

❑ Exercice 1 : QCM sur le système et le langage C

Dans cet exercice, chaque question peut avoir *zéro, une ou plusieurs bonnes réponses*. Pour chaque question, cocher la ou les cases correspondantes aux réponses exactes.

1. À propos du système de fichiers et des commandes Linux :
 - ☒ Le chemin `/home/alice/tp1` est un chemin absolu.
 - ☐ La commande `cd ..` permet de revenir dans le répertoire personnel de l'utilisateur.
 - ☒ La commande `ls -a` affiche tous les fichiers du dossier, y compris les fichiers cachés.
 - ☐ La commande `rm` permet de renommer un dossier ou un fichier.
2. Concernant les liens sous Linux :
 - ☐ Lorsqu'on crée un lien physique vers un fichier, on effectue une copie de ce fichier.
 - ☐ La commande `ln source.txt lien` crée un lien symbolique.
 - ☒ Supprimer le fichier `source` rend un lien symbolique orphelin (inutilisable).
 - ☐ Deux fichiers ne peuvent jamais avoir le même *inode*.
3. À propos des types de base en langage C :
 - ☒ Le type `int` permet de représenter des entiers relatifs.
 - ☐ Les chaînes de caractères possèdent le type natif `str`.
 - ☐ L'opérateur `**` permet d'élever un `float` à une puissance.
 - ☒ Les types `float` et `double` permettent tous les deux de représenter des nombres décimaux.
4. À propos des types et de la syntaxe en langage C :
 - ☒ L'inclusion de l'en-tête `<stdbool.h>` est nécessaire pour utiliser le type `bool`.
 - ☒ Une variable de type `char` s'écrit entre apostrophes simples (ex : `'a'`), tandis qu'une chaîne de caractères s'écrit entre guillemets (ex : `"a"`).
 - ☐ L'opérateur d'égalité en C s'écrit `=` et l'affectation s'écrit `:=`.
 - ☐ L'opérateur logique « et » s'écrit `||` et l'opérateur logique « ou » s'écrit `&&`.
5. Pour se déplacer dans son répertoire personnel, l'utilisateur `toto` peut taper :
 - ☐ `cd .`
 - ☒ `cd ~`
 - ☐ `cd ..`
 - ☒ `cd /home/toto/`
6. Que vaut la variable `x` après l'instruction `float x = 7 / 2;` en C ?
 - ☐ 3.5
 - ☒ 3.0
 - ☐ 3
 - ☐ 1.0
7. Quel affichage produit le fragment de code suivant ?

```
1  int s = 0;
2  for (int i = 1; i <= 4; i++) {
3      s += i;
4  }
5  printf("%d\n", s);
```

 - ☐ 4
 - ☐ 6
 - ☒ 10
 - ☐ 15
8. À propos des fonctions en langage C :
 - ☒ En C, le passage des arguments scalaires se fait par valeur (une copie est transmise).
 - ☐ Une fonction ne peut jamais avoir `void` comme type de retour.
 - ☐ Une fonction doit obligatoirement comporter au moins un argument.
 - ☐ Le mot-clé `return` est obligatoire dans le corps de toute fonction.

□ **Exercice 2** : *Ligne de commande et arborescence*

Bob travaille dans un sous-dossier **TP1** de son répertoire personnel. Ce dossier contient des fichiers source C (**.c**) et des fichiers de données texte (**.txt**). Écrire les commandes du shell qui lui permettent de réaliser les opérations suivantes :

1. Afficher le chemin absolu du répertoire de travail actuel dans la console.

```
pwd
```

2. Créer un nouveau sous-dossier nommé **src** dans le répertoire courant.

```
mkdir src
```

3. Déplacer tous les fichiers portant l'extension **.c** du dossier courant dans le sous-dossier **src**.

```
mv *.c src
```

4. Afficher les 10 premières lignes du fichier **data.txt** situé dans le répertoire courant.

```
head -n 10 data.txt
```

5. Rediriger la liste détaillée des fichiers du répertoire courant (avec **ls -l**) dans un fichier nommé **contenu.txt**.

```
ls -l > contenu.txt
```

6. Créer un lien symbolique nommé **raccourci.txt** dans son répertoire personnel (**~**) qui pointe vers le fichier **data.txt** du dossier courant.

```
ln -s ~/TP1/data.txt ~/raccourci.txt
```

7. En utilisant un tube (*pipe*) **|**, compter le nombre de fichiers et dossiers présents dans le répertoire courant (on rappelle que la commande **wc -l** compte le nombre de lignes reçues en entrée).

```
ls | wc -l
```

8. Afficher la 42^e ligne du fichier **prog.c** situé dans le répertoire courant.

```
head -n 42 prog.c | tail -n 1
```